Cloud Service Models

how much of the stack do you want to own?

Watch the 4-min video

In one line: IaaS, PaaS, FaaS and SaaS differ in **how much of the stack the provider runs**: the further you move toward SaaS, the less you patch, scale and operate, and the less you control; under the **shared responsibility model** your data, identities and configuration are always yours.

1 Three engineers, one database

The team needs a relational database. One engineer wants SQL Server on virtual machines for full control. Another wants Azure SQL Database so Microsoft handles patching and failover. A third asks whether a SaaS product could replace the custom app altogether. All three are valid; they differ in which layers of the stack you own, and therefore what you pay for in money versus engineering time.

2 The models, layer by layer

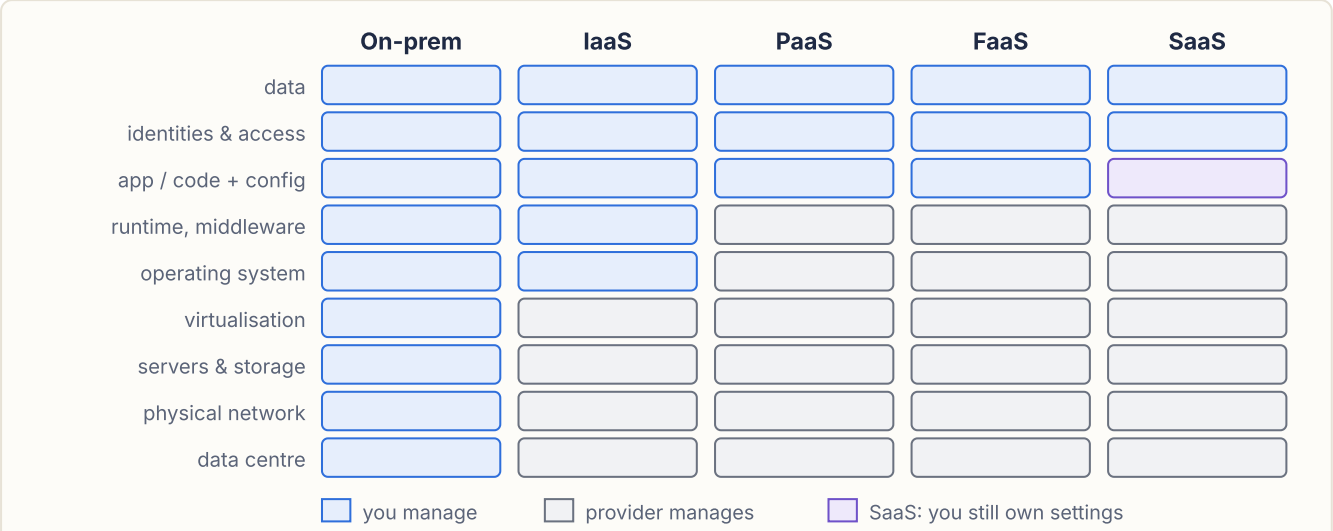


Figure 1. Who manages each layer. Data, identities and access are always yours; the provider always owns the physical layers in the cloud.

- **IaaS** is the closest to your own data centre: you install and patch the OS and software, configure backups and high availability, and size the machines. Maximum control and portability, maximum operational work.
- **PaaS** hands over the OS, runtime and much of operations: patching, backups, HA, often scaling. You bring code, data and configuration and accept the service's limits (versions, extensions, settings).
- **FaaS / serverless** goes further: deploy functions, pay per execution, never see a server. Limits shape the design: AWS Lambda runs at most 15 minutes per invocation; Azure Functions on the Consumption plan time out after 5 minutes by default (10 max). Expect **cold starts** when scaling from zero. 'Serverless' also describes PaaS that bills per use, like serverless database tiers.
- **SaaS**: you configure and use; your job is users, access, data and settings.

Model	You get	Examples	You pay for
IaaS	VMs, disks, networks	Azure VMs, AWS EC2, Google Compute Engine	provisioned capacity per second/hour, whether busy or not
CaaS (between)	a managed container platform	AKS, EKS, GKE; Azure Container Apps, Cloud Run, Fargate	nodes, or vCPU/memory per second
PaaS	a managed platform: DB, app host, queue	Azure SQL, App Service, RDS, Cloud SQL, Service Bus	service tier (vCores, DTUs, instance size)
FaaS	functions run on events, scale to zero	Azure Functions, AWS Lambda, Cloud Run functions	invocations + execution time × memory
SaaS	a finished application	Microsoft 365, Salesforce, GitHub	per user or per usage, monthly

3 The shared responsibility model

The provider is responsible for **security of the cloud**: physical data centres, hardware, the host network and the virtualisation layer. You are responsible for **security in the cloud**. Which layers that covers depends on the model (Figure 1), but some are **always yours**: your data and its classification, your identities and access (who has admin rights, MFA, keys), your endpoints, and how you configure the service (public network access, firewall rules, encryption settings, backups retention). Most cloud breaches come from customer-side misconfiguration, such as a storage container left public or an over-privileged key in a repo, not from the provider's infrastructure.

4 When to self-manage

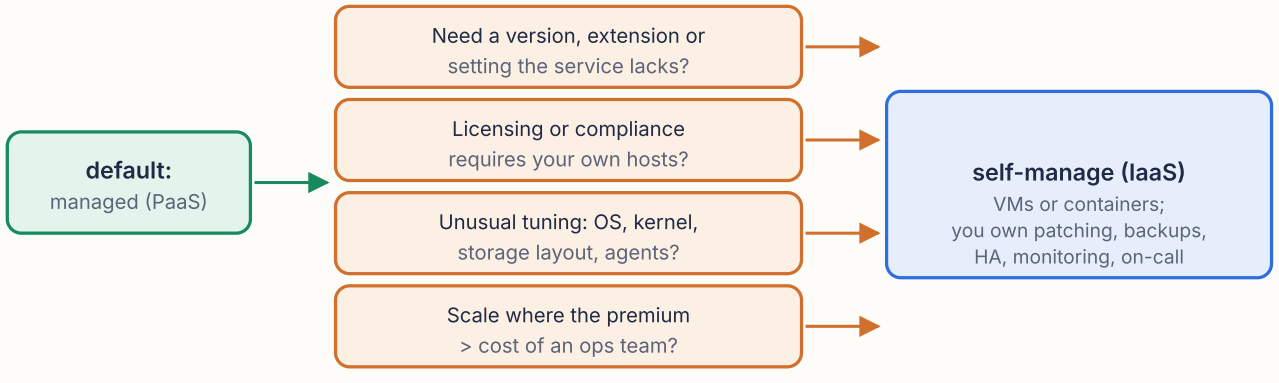


Figure 2. Start from managed services and require a concrete reason to take on the operations yourself.

Managed services usually win because engineering time is expensive and patching, failover and backups are undifferentiated work. Reasons to self-manage are specific: a database extension or version the service doesn't offer; software that must run with OS-level agents or custom kernels; licensing that ties you to your own hosts; compliance demanding dedicated hardware (though dedicated-host options exist); or a very large, steady footprint where the managed premium exceeds the cost of a team that runs it well. Also count **lock-in**: PaaS APIs and features differ across providers, while VMs and containers move more easily. A middle path is often best, e.g. Azure SQL Managed Instance (near-full SQL Server compatibility, still managed) or SQL Server on a VM with the provider's automation extension for patching and backups.

WORKED EXAMPLE: AZURE SQL (PAAS) VS SQL SERVER ON VMS (IAAS)

Task	SQL Server on VMs	Azure SQL Database
OS + SQL patching	you schedule and test	automatic, in maintenance windows
Backups	you configure, store and test restores	automatic; point-in-time restore, 1–35 days (default 7)
High availability	you build Always On availability groups / failover cluster	built in; zone-redundant option
Scaling	resize VMs, re-plan storage	change tier or vCores; serverless tier can auto-pause
Control	full: any version, agent, setting	limited: no OS access, some features differ

The team's app uses standard SQL features. Azure SQL costs somewhat more than the raw VMs, but replaces most of a DBA's routine work (patching, backup jobs, failover drills) and comes with a financially backed SLA, so they choose PaaS and keep the VM option for a legacy system that needs SQL Server Agent jobs and cross-database queries, which they move to Managed Instance instead.

TRADE-OFFS

- **Control vs operations**: every layer you own is a layer you patch, monitor and get paged for.
- **Unit price vs total cost**: VMs look cheaper per hour; PaaS often wins once people's time is counted.
- **Portability vs features**: managed services are convenient but provider-specific.
- **FaaS**: no idle cost and automatic scale, but time limits, cold starts and per-request cost at high steady load.

COMMON MISTAKES

- Assuming 'managed' means the provider secures your data and access settings too.
- Comparing VM price to PaaS price without counting engineering and on-call time.
- Putting long-running or steady high-throughput work on FaaS and paying more than a container would cost.
- Choosing IaaS for 'control' without a concrete feature or requirement that needs it.

5 Key terms

IaaS: rent VMs, disks, networks; you run the OS up

PaaS: provider runs OS and platform; you bring code and data

FaaS: event-driven functions, pay per execution

SaaS: finished application; you manage users and data

Shared responsibility: provider secures the cloud, you secure what's in it

Managed premium: extra price for operations you don't do

6 Interview questions

Q1. When would you self-manage instead of using a managed service? BASIC

When you need control, features or versions it lacks, when licensing or compliance demands it, or at a scale where the managed premium exceeds your operational cost.

Q2. What is the shared responsibility model? BASIC

The provider secures the underlying cloud; you always own your data, identities and configuration, with more layers shifting to the provider from IaaS to SaaS.

Q3. What's the difference between PaaS and FaaS? BASIC

PaaS runs your app on a managed platform that is usually always on; FaaS runs individual functions per event, scales to zero and bills per execution, with time limits and cold starts.

Q4. A startup's API has spiky traffic and idle nights. Which model? SCENARIO

FaaS or a scale-to-zero container platform for the API, with managed databases; it avoids paying for idle capacity, as long as cold starts and time limits are acceptable.

Q5. Your PostgreSQL needs an extension the managed service doesn't support. Options? SCENARIO

Check whether the provider's other tiers or flexible-server options support it, redesign around it, or self-manage on VMs/Kubernetes and accept the operational load.

Q6. A storage bucket on a managed service was leaked. Whose fault under the shared model? TRICKY

Usually the customer's: access configuration is always the customer's responsibility, even on fully managed services.

Q7. Is serverless always cheaper? TRICKY

No: at steady high utilisation, per-request pricing can exceed reserved VMs or containers; serverless wins on spiky or low traffic and on saved ops time.

Next: SD-V-002 · Regions and Availability Zones backendroadmap.com